DCNNet: A Distributed Convolutional Neural Network for Remote Sensing Image Classification

Ting Zhang, Zhirui Wang, *Member, IEEE*, Peirui Cheng, Guangluan Xu, and Xian Sun, *Senior Member, IEEE*

***Abstract*— With the development of information technology, multiplatform collaborative collection and processing of remote sensing (RS) images has become a significant trend. However, the existing models are challenging to achieve accurate and efficient image interpretation on RS multiplatform systems. To solve this problem, we propose a novel distributed convolutional neural network (DCNNet) and demonstrate the superiority of our method in RS image classification. First, a progressive inference mechanism is introduced to support most images to be classified in advance with satisfactory accuracy, which minimizes redundant cloud transmission and achieves higher inference acceleration. Meanwhile, a distributed self-distillation paradigm is designed to integrate and refine in-depth features, performing efficient knowledge transfer between the terminals and the cloud network. Second, a multiscale feature fusion (MSFF) module is presented to extract valid receptive fields and assign weights to crucial channel dimension features. Finally, a sampling augmentation (SA) attention is proposed to enhance the effective feature representation of RS images through a bottom-up and top-down feedforward structure. We conducted extensive experiments and visual analyses on three benchmark scene classification datasets and one fine-grained dataset. Compared with the existing methods, DCNNet consolidates several advantages in terms of accuracy, computation, transmission, and processing efficiency into a single framework for multiplatform RS image classification.**

***Index Terms*— Attention mechanism, distributed network, progressive inference, remote sensing (RS) image classification, self-distillation.**

NOMENCLATURE

x	The input image.
$\hat{y}$	The predicted label of x.
$\mathcal{V}$	The number of branches.
$\hat{e}^v$	The confidence entropy score of $\hat{p}^v$.
$f^v_{exit}(\cdot)$	The last layer of branch v.
z^v	The output of $f^v_{exit}(\cdot)$.
τ	Distillation temperature.
F	Feature map of the backbone.
W, H, C	Width, height, and channel number of F.
C^*	The number of convolution kernels for F'_i.
S'_i	The output feature of F'_i in PSC module.
Z'_i	The channel-weighted output of S'_i.
m	The kernel size of 1D convolution.
$\mathcal{L}_{FL}$	Multiclass Focal Loss function.
$\mathcal{L}_{L2}$	Least squared error loss function.
$a \cdot b$	The inner product of a and b.
y	The one-hot ground-truth of x.
J	The number of classes.
$\hat{p}^v$	Probability vector of x on the v^{th} branch.
$\mathcal{E}^v$	Pre-defined confidence entropy threshold.
$f^v_{extract}(\cdot)$	Feature extractor to branch v.
$\hat{x}^v$	The input feature to branch v.
γ	Focusing parameter.
$\mathcal{G}_v$	Feature map of the v^{th} branch.
F'_i	The i^{th} splited part of F.
P_i	The padding size of F'_i in convolution.
$\mathcal{W}_{S'_i}$	The channel-wise weight of S'_i.
Z_{out}	The output feature of F in MSFF module.
k_i	The kernel size of 2D convolution.
$\mathcal{L}_{KL}$	Kullback-Leibler divergence loss function.
μ, β	The coefficients to balance $\mathcal{L}_{KL}$ and $\mathcal{L}_{L2}$.
$a \circ b$	The element-wise product of a and b.

Manuscript received 25 October 2022; revised 30 December 2022; accepted 1 February 2023. Date of publication 8 February 2023; date of current version 24 February 2023. This work was supported by the National Nature Science Foundation of China under Grant 61725105, Grant 62171436, and Grant 62076241. *(Corresponding author: Xian Sun.)*

Ting Zhang and Xian Sun are with the Aerospace Information Research Institute and the Key Laboratory of Network Information System Technology (NIST), Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100190, China, also with the University of Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Electronic, Electrical and Communication Engineering, University of Chinese Academy of Sciences, Beijing 100190, China (e-mail: zhangting20@mails.ucas.ac.cn; sunxian@aircas.ac.cn).

Zhirui Wang, Peirui Cheng, and Guangluan Xu are with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China, and also with the Key Laboratory of Network Information System Technology (NIST), Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100190, China (e-mail: zhirui1990@126.com; chengpr@aircas.ac.cn; gluanxu@mail.ie.ac.cn).

Digital Object Identifier 10.1109/TGRS.2023.3243238

I. INTRODUCTION

AS THE fundamental research of remote sensing (RS) interpretation, RS image classification has aroused great interest due to its comprehensive and valuable applications, such as geospatial object detection [1], [2], natural disaster detection [3], land resource management [4], [5], and environmental monitoring [6]. With the rapid progress of technology, the development of RS platforms is becoming mature and extensive. The collection and processing of massive RS images are not limited to a single device but through the cooperation of multiple platforms. In addition, the RS application raises high requirements for the reliability and real-time performance of networks configured on various devices. As a well-suited

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

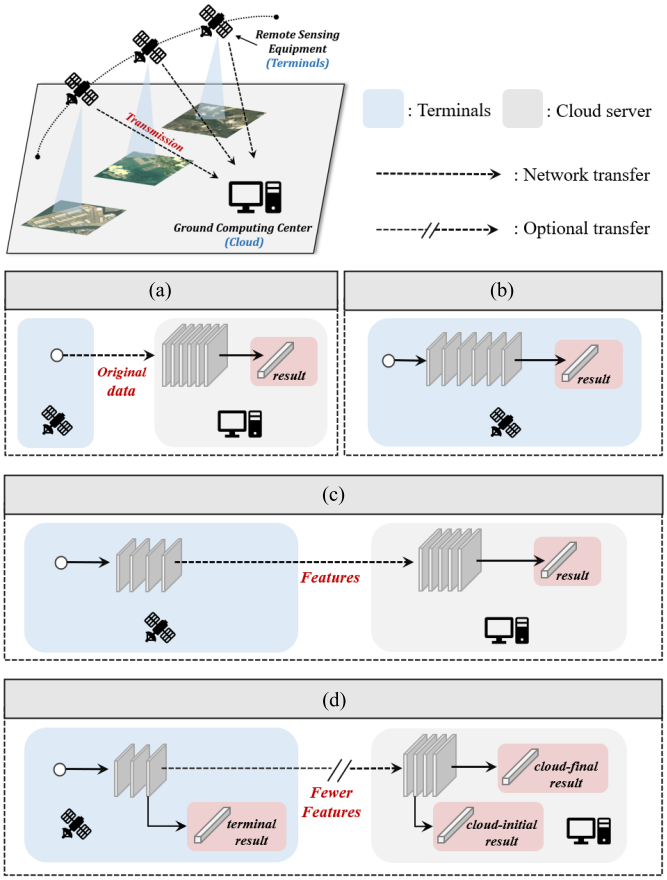

Fig. 1. Comparison between our distributed framework and existing methods. (a) Centralized methods (cloud-only). (b) Decentralized methods (terminal only). (c) Traditional distributed methods. (d) Our distributed framework:DCNNet.

multiplatform processing method, the distributed model can fully use the computing resources of the terminals (e.g., satellites) and cloud (e.g., ground computing center), reducing response latency and providing an accurate and efficient choice.

The traditional RS image classification mode is shown in Fig. 1(a). It uniformly transmits the collected RS data to a ground computing center and uses a large neural network for centralized classification. However, this centralized mode obviously cannot meet the standards of low latency and fast response in RS applications. With the continuous emergence of lightweight methods, a popular alternative is directly implementing image classification on terminals. As presented in Fig. 1(b), these methods can be collectively classified as the decentralized mode. Compared with the centralized mode, the decentralized mode greatly optimizes response efficiency. Nevertheless, it has poor accuracy and limited practical application capabilities for challenging RS image classification due to the terminal's lower computing power and energy consumption constraints [7], [8], [9].

The distributed model [10], [11], [12], [13] provides a better multiplatform solution than the above two models. As shown in Fig. 1(c), the distributed mode splits sections of neural networks into a distributed hierarchy, deploys the networks with smaller parameters on the terminals, and configures the remaining larger network on the cloud with sufficient computing resources. The smaller network on each terminal can quickly perform initial feature extraction and pass features to the cloud for further processing and classification. Therefore, the distributed structure has the advantage of high efficiency and low transmission costs compared with the centralized mode, which constantly offloads the original data to the cloud. Meanwhile, distributed methods can achieve higher accuracy with terminal-cloud cooperation than directly processing on decentralized terminals.

However, there are still some critical challenges in multiplatform RS classification. The practical application of RS has poor communication conditions and involves long-distance transmission, so the transmission costs cannot be ignored. Although some distributed methods replace the original data by passing processed features [14], [15], the frequent transmission of each data is still unable to meet the fast response requirement. Meanwhile, most existing methods rely on the cloud to generate output, which may suffer from unstable connectivity conditions and the uncertain availability of the cloud. In addition, unlike natural images, RS images have extensive geographic coverage, more ground object categories, and complex structures. Under the influence of diverse backgrounds information, the classification of RS images is more likely to be disturbed by different geographical features. At the same time, the inter-class similarity of fine-grained ground objects and the unique vertical imaging perspective of RS images also affect the performance of many models. Therefore, further improving the accuracy of RS image classification in the distributed structure is also a significant problem.

According to the above problems, this article proposes a distributed convolutional neural network (DCNNet) for multiplatform RS image classification. First, as presented in Fig. 1(d), we reformulate a new distributed architecture which takes advantage of a progressive inference mechanism. This approach allows the overall framework to exit early at different parts of the neural network during inference based on the complexity of inputs. Therefore, when the terminal obtains confident prediction, the classification results can be output directly without transmission to the cloud, minimizing redundant transmission and weakening over-reliance on the cloud model. During training, DCNNet introduces a novel distributed self-distillation paradigm adapted to each classification branch. The deep semantic information of the cloud's feature extractor supervises the training of terminal models, effectively improving the terminal model's convergence speed and accuracy. Furthermore, the effective combination of the self-distillation paradigm and progressive inference gives input images more opportunities to obtain satisfactory classification results on terminals. Secondly, we propose a multiscale feature fusion (MSFF) module to address the classification challenge of complex RS images. The MSFF module extracts multiscale effective receptive fields through a pyramid separable convolution (PSC) module and introduces cross-channel attention to provide interactive guidance. With the contribution of the MSFF module, DCNNet effectively improves the accuracy performance of RS image classification. Finally, we design a sampling augmentation (SA) attention on each classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

branch, which brings discriminative feature representation through a fast bottom-up feedforward process and top-down attentional feedback. SA attention effectively optimizes the processing ability of classification branches with few parameters and computation.

The main contributions of this article are summarized below.

1) This article proposes a novel distributed framework for multiplatform RS image classification. Our DCNNet combines a progressive inference mechanism and distributed self-distillation paradigm, significantly improving classification accuracy and optimizing comprehensive inference efficiency.
2) An MSFF module is presented, which provides long-term supervision to multiscale feature information, achieving satisfactory classification accuracy and better robustness in RS classification tasks.
3) To enhance the feature representation of RS images on each classifier, this article designs an SA attention that increases the proportion of input images classified from terminals and reduces redundant transmission.

The rest of this article is organized as follows. Section II introduces related works. Section III presents the proposed DCNNet method. Section IV reports the experimental results and conducts a comparative analysis. Eventually, Section V gives the conclusion.

II. Related Work

A. RS Image Classification

In earlier works, hand-crafted models are the mainstream methods in image classification [16], [17], [18]. However, with the rapid progress of convolutional neural networks (CNNs) in visual understanding tasks, CNN-based models have become popular and achieved great success [19], [20], [21], [22], [23], [24], [25], [26]. These CNN-based models are able to learn features from input images using a hierarchical structure consisting of convolutional layers, pooling layers, and FC layers. AlexNet, designed by Krizhevsky et al. [27], won first place in the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2012). Simonyan and Zisserman [28] in 2014 proposed a Visual Geometric Group Network (VGG16) with 16 layers. In 2015, Castelluccio et al. proposed a fine-tuning technique for CaffeNet and GoogLeNet [29] using RS data for land-use classification. With the increasing depth of the structure, He et al. [30] designed a residual network (ResNet) in 2016. Zhang et al. [31] fully used the advantages of CNN and CapsNet models to encode features' attributes and spatial information in RS images. Chen et al. [32] introduced a context-preserving architectural learning network for RS scene classification. Li et al. [33] proposed a stochastic topology and multiscale mapping method to develop an RS image recognition architecture.

In RS images, attention mechanisms have been widely used in CNN methods due to the importance of spatial regions and channel dimensions in feature maps [34], [35]. Wang et al. [36] designed an end-to-end ARCNet for RS scene classification. The classifier mainly focuses on critical regions like the human visual system with the recurrent attention structure. Fang et al. [37] proposed a spectral-based attention method for hyperspectral samples, which utilizes dense CNNs to extract features. Zhu et al. [38] proposed an attention-based deep feature fusion (ADFF) model that combines the original image and Grad-CAM attention map to distinguish repetitive surfaces and salient areas. Bi et al. [39] proposed an APDC-Net framework. The backbone consists of a dense connection structure of the attention pool and presents a multilevel supervision method. Zhang et al. [40] combined graph convolutional networks to capture nonlocal topological relations. The proposed method optimizes the dynamic update process to achieve more accurate feature embedding. Zhang et al. [41] proposed a structural optimization transmission framework for multisource images, which utilizes cross-attention to enhance critical features effectively. Although the above methods perform well in RS classification, they do not consider the requirements of multiplatform processing applications.

B. Methods to Improve the Efficiency of CNNs

Lightweight Models: The lightweight model alleviates the inefficiency of the overparameterized model. Google presented a class of efficient models called MobileNet. MobileNetv1 [42] employs depthwise separable convolutions to construct lightweight neural networks. MobileNetv2 [43] designed an inverted residual with the linear bottleneck. Built upon MobileNetv2, MobileNetv3 [44] combined NAS and NetAdapt algorithms to search for the optimal model structure. The remaining representative lightweight models include Xception [45], SqueezeNet [46], and ShuffleNet [47], providing innovative ideas for models on mobile devices. On this basis, Cui et al. [48], inspired by the structure of MobileNetV1, proposed a lightweight CNN classification method based on branch feature fusion (LCNN-BFF), which integrates depthwise separable convolution into classic convolution to realize feature extraction. Yu et al. [49] proposed BiMobileNet, which uses MobileNetv2 to extract in-depth features and combines feature fusion with bilinear pooling.

Knowledge Distillation: Knowledge distillation is a method to compress an overparameterized teacher model into a compact student model, enabling the student model to achieve significant performance gains. In 2015, Hinton et al. [50] first introduced a method for knowledge distillation by transferring the output logit of the teacher network to the student network and applying it to the classification task. Zagoruyko et al. [51] first proposed establishing an attention distillation loss in the corresponding stages of the teacher model and student model. Furthermore, self-distillation, as a unique strategy of knowledge distillation, achieves efficient knowledge transfer without training an additional teacher model. Hou et al. [52] introduced a self-attention distillation (SAD) method. Zhang et al. [53] proposed a group of auxiliary weak classification structures according to the hierarchy of the backbone architecture and used the features of the intermediate hidden layer to classify the output. Ji et al. [54] proposed a self-knowledge distillation network architecture based on top-down and bottom-up paths.

Early-exiting Methods: The early-exiting method helps the samples to output at the appropriate stage adaptively, which

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

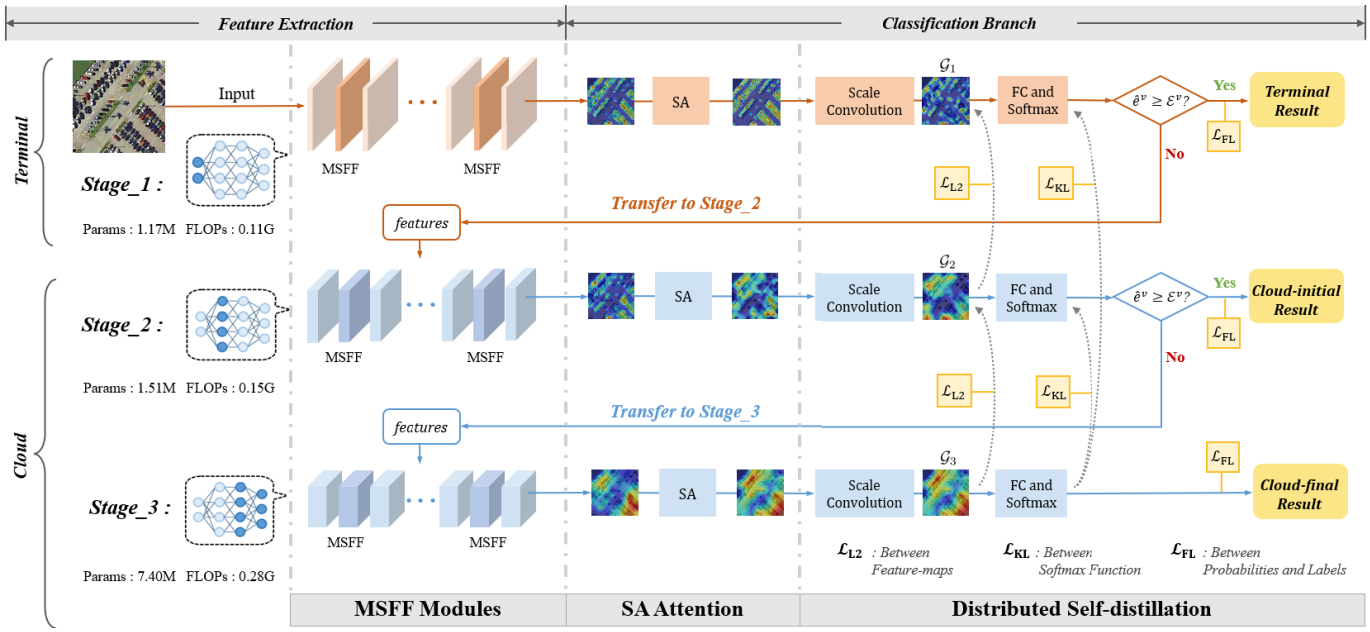

Fig. 2. Overall architecture of the proposed method. (i) Whole framework can be divided into three parts: *Stage_1*, *Stage_2*, and *Stage_3*. (ii) *Stage_1* is deployed in the terminal, *Stage_2* and *Stage_3* are configured in the cloud. (iii) Each stage consists of a feature extractor and a classification branch, allowing to receive features from the previous stage or output results when achieving thresholds. (iv) Proposed MSFF module is the central unit of feature extraction. SA attention and a scale convolution block compose each classification branch. Distributed self-distillation combined with $\mathcal{L}_{FL}$, $\mathcal{L}_{KL}$, and $\mathcal{L}_{L2}$ to achieve joint training.

provides an innovative method for improving the efficiency of the neural network. Adaptive computation time proposed using a trainable stopping strategy to perform inference [55], [56]. BranchyNet [57] proposed a new network structure that added side-branch classifiers and used confidence entropy to infer samples. FastBERT [58] and DeeBERT [59] combined ideas from BranchyNet on pretrained language models. RightTool [60] adopted the same early-exiting criteria as shallow-deep network, while Zhou et al. [61] proposed the patience-based early exit (PABEE) method. We further explore the unique potential of the early-exiting method and introduce it into the distributed architecture, propose progressive inference in the distributed hierarchy, and optimize the transmission and responsiveness of the whole network.

C. Distributed Computing Models

With the rapid development of information technology, the framework of large-scale distributed computing system is of great significance in the era of multiplatform interconnection. Most existing distributed models are federated learning methods, such as FedAvg [10] and FedMA [11], which have great challenges in training large neural networks on resource-constrained devices. Recent works [62] and [12] work on large CNNS but rely on GPU training to accomplish the evaluation. In addition, work [14], [15] optimizes the communication cost but ignores the terminal model's computational resource and power consumption constraints. The work [13] proposes to map the various sections of the neural network to the distributed computing hierarchy. However, the terminal processing capability of this method is weak, and the coordination of the cloud-terminal model is not considered. We comprehensively consider the possible problems of deploying traditional distributed methods directly to RS applications. Then, focusing on the innovation of the model structure, the proposed DCNNet achieves better improvements in accuracy, efficiency, computation, and transmission for multiplatform RS classification tasks.

III. Proposed Method

A. Overview

The overall framework of the proposed method is shown in Fig. 2. DCNNet uses a lightweight structure and refines the overall network into three stages to adapt to distributed requirements. Each stage consists of a feature extraction network with MSFF modules and a classification branch. With the deepening of the feature extraction network from *Stage_1* to *Stage_3*, the corresponding classification accuracy is gradually improved. *Stage_1* is deployed as the terminal model and the *Stage_2* to *Stage_3* are configured in the cloud. We combine the progressive inference mechanism based on the distributed hierarchical structure. After inputting the collected data into the network, the *Stage_1* classifier first judges whether it meets the confidence entropy threshold. If the image's confidence reaches the threshold, the result can be output directly on the terminal. If not, the image is passed to cloud in the form of feature data for further processing. Subsequently, the remaining model on the cloud can still take advantage of the progressive inference mechanism, which inserts the classification branches of *Stage_2* and *Stage_3* to achieve early-exiting, reducing the computation and pursuing efficient inference acceleration. During training, we incorporate a novel distributed self-distillation paradigm composed of three parts. The first is the multiclass focal loss (FL) between labels and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

each branch. The second is the Kullback–Leibler divergence distillation of probability distributions between the softmax layers at each stage. The last part is the distillation between feature maps of the same dimension after scale convolution modules.

In the process of network construction, we propose an MSFF module and SA attention for the feature extractor and classifiers, respectively. Firstly, the feature extractor of each stage is composed of several bottleneck blocks with MSFF modules as central units. The proposed MSFF module effectively integrates cross-dimensional feature information to improve the classification performance of the overall distributed model. Secondly, an SA attention is designed for each classification branch to generate attention-aware features and improve the accuracy with few parameters and computation. The above two modules effectively improve the accuracy performance of DCNNet distributed network for various RS image classification tasks. In addition, the notations and corresponding descriptions involved in the method are shown in the Nomenclature.

B. Progressive Inference

To better realize the trade-off between the accuracy and efficiency of DCNNet, we introduce the progressive inference mechanism that allows the fast and reliable execution of inference across terminals and the cloud. Specifically, easy-to-classify images are allowed to exit in terminals (*Stage_1*) when achieving a confident early prediction. The remaining images that fail to meet the required threshold at the terminal will be transmitted to the cloud (*Stage_2*, *Stage_3*) in the form of feature maps. The deep network in the cloud can further extract the semantic features of RS images, contributing to more accurate predictions. Therefore, many samples can be classified in advance at the terminal while ensuring accuracy, significantly reducing redundant transmission to the cloud. At the same time, by combining with distributed classification branches, terminals can independently complete classification tasks even if the connection is unstable or the cloud is unavailable. Furthermore, when the input images are transmitted to the cloud, some of them can be confidently classified by *Stage_2* without passing to *Stage_3*. Thus, progressive inference in the cloud can also take advantage of the early-exiting [57], [59] to achieve faster inference acceleration. We utilize the probability vectors output by the fully connected and softmax layers to judge whether test images satisfy the exit condition. This judgment condition can be measured by normalized entropy, which represents the uncertainty of the test samples. Therefore, the confidence entropy score is calculated as

$$\hat{e}^v = 1 - \sum_{j=1}^{J} \frac{\hat{p}_j^v \log \hat{p}_j^v}{\log 1/J}. \tag{1}$$

Here, $\hat{p}_j^v$ is the probability that the input image x belongs to class j on the vth branch, and J is the number of classes. The value of $\hat{e}^v$ ranges from 0 to 1, and $\hat{e}^v$ close to 1 indicates that the network is confident enough for the test image. The predefined confidence entropy threshold of the vth branch is set to $\mathcal{E}^v$ (except for the deepest branch). The inference process starts from the terminal exit point and then iterates to the cloud exit point of the network. If $\hat{e}^v \geq \mathcal{E}^v$, the model returns the class label with the largest probability. Otherwise, the network has no confidence in the results and will enter deeper branches for judgment. The network will always return the label with the maximum score if the sample reaches the deepest exit point. For clarity, the progressive inference process of DCNNet is shown in Algorithm 1.

Algorithm 1 Progressive Inference

Input:

x: the input data of the mode

J: the set of all possible labels

$\mathcal{V}$: number of classification branches

$\mathcal{E}^v$: pre-defined confidence entropy threshold of the vth branch

Output:

$\hat{y}$: the predicted label of x

Procedure:

Load model parameters;
$\hat{y} = None$;
$\hat{x}^0 = x$;
for v from 1 to $\mathcal{V} - 1$ **do**
  Calculate the probability vector $\hat{p}^v$:
  $\hat{x}^v = f_{extract}^v(\hat{x}^{v-1})$;
  $z^v = f_{exit}^v(\hat{x}^v)$;
  $\hat{p}^v = softmax(z^v) = \frac{exp(z^v)}{\sum_j^J exp(z_j^v)}$;
  Calculate the confidence entropy score $\hat{e}^v$ of $\hat{p}^v$ according to Eq. (1)
  if $\hat{e}^v \geq \mathcal{E}^v$ **then**
    $\hat{y} = \arg\max\ (\hat{p}^v)$, Break;
  end if
end for
if $\hat{y}$ *is* $None$ **then**
  $\hat{y} = \arg\max\ (\hat{p}^{\mathcal{V}})$;
end if

C. Distributed Self-Distillation

We propose a distributed self-distillation paradigm for the joint training of multiple classification branches, which better integrates and refines network advantages of different depths. During joint training, the probability vector $\hat{p}^v$ with distillation temperature τ is computed as

$$\hat{p}^v = \frac{\exp(z^v/\tau)}{\sum_j^J \exp(z_j^v/\tau)} \tag{2}$$

where z^v is the output of x after fully connected layers, $v = 1, 2, 3$ represent the three branches in the *Stage_1*, *Stage_2*, and *Stage_3*. A larger τ makes the probability distribution $\hat{p}^v$ closer to smoothness [50].

Knowledge distillation first generates a teacher model and then orients the compact student model to an approximate over-parameterized teacher model. Unlike traditional knowledge distillation, self-distillation enables mutual guidance

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

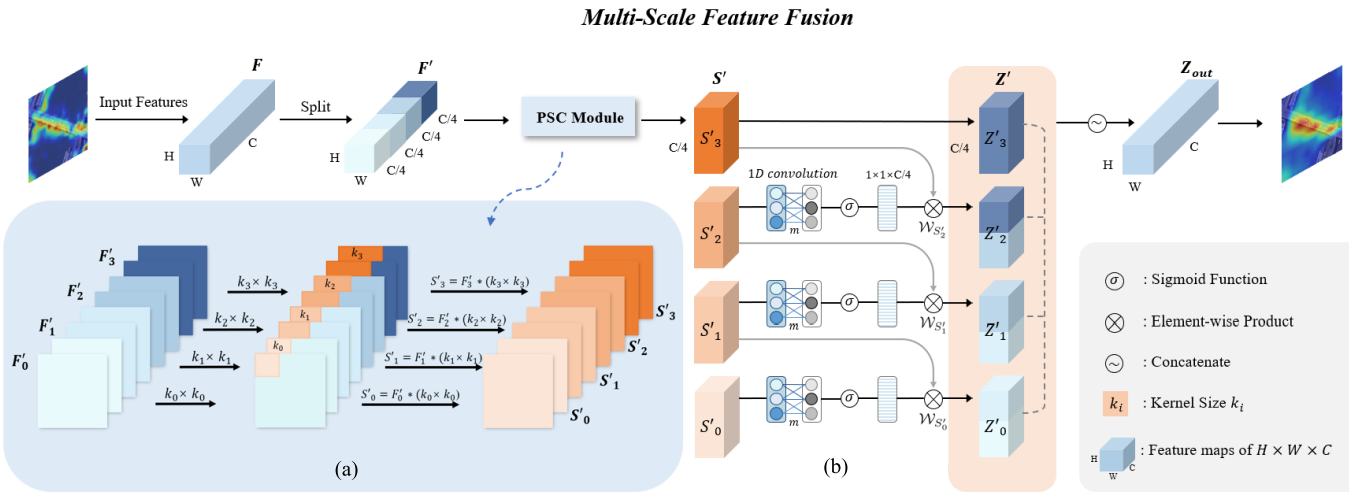

Fig. 3. Diagram of the MSFF module. It is composed of a PSC module and a cross-channel attention. Here, the number of splits n is equal to 4. (a) PSC. (b) Cross-channel attention.

between different layers in its own network without training an additional teacher model [53]. In DCNNet, taking advantage of the joint training in the three-stage branches, the valuable contextual information of the deep network is used as supervision, allowing a hierarchical top-down knowledge transfer. The branch of *Stage_3* is represented as the teacher model, and the branches of *Stage_1* and *Stage_2* are denoted as student models. The proposed self-distillation joint loss can be divided into three parts, and the first is the multiclass FL from labels to each branch, that is.,

$$\mathcal{L}_{FL} = \sum_{v=1}^{\mathcal{V}} -(1 - y \cdot \hat{p}^v)^{\gamma} \log(y \cdot \hat{p}^v) \tag{3}$$

where $\hat{p}^v$ is the probability vector on the vth branch. y is the one-hot ground truth. $y \cdot \hat{p}^v$ represents the inner product of y and $\hat{p}^v$, which means the probability value corresponding to the true label. The modulation factor $(1 - y \cdot \hat{p}^v)^{\gamma}$ is used to increase the loss contribution of samples with low probability vector [63], and γ is the focusing parameter. $\mathcal{L}_{\mathrm{FL}}$ calculates the error between predicted probabilities and ground truth so that the true labels in the dataset directly provide supervision for each branch.

The second supervision is the Kullback–Leibler divergence, which utilizes the deepest teacher model to guide the training of shallow branches. By computing the softmax output of each branch smoothed with distillation temperature, the shallow branches fit the predicted distribution of the deepest branch to obtain richer supervision information. Formally, the Kullback–Leibler divergence is defined as

$$\mathcal{L}_{\mathrm{KL}} = \mathrm{KL}(\hat{p}^v \,||\, \hat{p}^{\mathcal{V}}) = \sum_{v=1}^{\mathcal{V}-1} \sum_{j=1}^{J} \hat{p}_j^{\mathcal{V}} \log(\hat{p}_j^{\mathcal{V}} / \hat{p}_j^v). \tag{4}$$

The third part of supervision is establishing an L2 distillation loss between the feature maps after each classifier scale convolution module. The L2 loss implements a top-down attention distillation between feature maps of adjacent branches [52]. Specifically, the feature map $\mathcal{G}_1$ of *Stage_1* imitates the feature map $\mathcal{G}_2$ of *Stage_2* to strengthen feature representation, and the feature map of *Stage_2* is also supervised by the deepest one $\mathcal{G}_3$ of *Stage_3*. Meanwhile, the feature representations learned by the shallow branches in turn benefit the deep branches. Note that the feature maps after the scale convolution module of the three branches have the same dimension. The formula can be written as

$$\mathcal{L}_{L2} = || \mathcal{G}_2 - \mathcal{G}_1 ||_2^2 + || \mathcal{G}_3 - \mathcal{G}_2 ||_2^2 . \tag{5}$$

Conclusively, the loss function of the entire neural network consists of a partial merge of each classifier. The overall loss function is formulated by

$$\mathcal{L} = \mathcal{L}_{FL} + \mu \mathcal{L}_{KL} + \beta \mathcal{L}_{L2} \tag{6}$$

where μ and β are the coefficients to balance two terms. The DCNNet is optimized by reducing the total loss objective function as (6). It should be noted that for the classification branch of *Stage_3*, the values of these two parameters are zero, since the supervision of the deepest classifier comes only from the labels.

D. Multiscale Feature Fusion Module

Different from natural images, RS images have complex backgrounds and rich hierarchical features, which are more susceptible to noise interference and difficult to classify. To improve the accuracy of DCNNet feature extraction more comprehensively without ignoring image details, we propose the MSFF module as the main component of the feature extractor at each stage. MSFF module is used to replace the 3 × 3 convolution of the bottleneck block to extract features. Unlike existing methods, the MSFF module establishes long-term channel dependencies, effectively integrating local and global attention. At the same time, it combines the depthwise separable convolution and the group convolution to reduce the model complexity, enabling the overall distributed network to have a lightweight structure while ensuring classification accuracy.

As described in Fig. 3, the MSFF module consists of a PSC module and a cross-channel attention. It is mainly implemented in three steps. First, we utilize the PSC module to extract channel-wise multiscale features. Second, feature

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

maps at different scales are fused and calibrated using the cross-channel attention, and element-wise product operations are applied to each set of scales. Finally, we concatenate the multiscale channels to obtain a refined feature map with richer information.

As illustrated by Fig. 3(a), the essential operator for implementing the multiscale feature extraction in the proposed MSFF module is the PSC module. The input feature $F \in \mathbb{R}^{H\times W\times C}$ is split into n parts as $\left[F'_0, F'_1, \ldots, F'_{n-1}\right]$ along the channel direction, and the ith feature map is $F'_i \in \mathbb{R}^{H\times W\times C/n}$ with $i = 0, 1, \ldots, n-1$. $\left[k_0, k_1, \ldots, k_{n-1}\right]$ are denoted as convolution kernels of different scales. For each F'_i, we correspondingly employ the convolution kernels of scale k_i to perform depthwise separable convolution, where the number of convolution kernels for each part is $C^* = C/n$. The reason for using depthwise separable convolutions is to reduce parameters and FLOPs in multiscale computations effectively. In the PSC module, each split part F'_i is able to independently learn the multiscale spatial information. Meanwhile, the PSC module can process input features at multiple scales in parallel, resulting in different spatial resolutions. Accordingly, the multiscale feature map generation function can be written as follows:

$$S'_i = \text{Conv}[(k_i \times k_i, C^*, P_i) * (F'_i)] \tag{7}$$

where $S'_i \in \mathbb{R}^{H\times W\times C^*}$ denotes the ith group of feature maps with multiscale information extracted, and $P_i = (k_i - 1/2)$ is the padding size corresponding to each group of convolution kernels.

The attention mechanism has been demonstrated to offer significant potential in improving the performance of classification networks. Therefore, based on the multiscale features of the pyramidal separable modules, we propose the cross-channel attention to collaboratively refine S'_i. As shown in Fig. 3(b), the cross-channel attention ensures that features at each scale can acquire complementary information from other related channels to enrich themselves. This enables the model to focus on global features and details in complex contexts. Specifically, we assume that the size of the convolution kernel represented by k_i grows with the subscript i, where the ith kernel size $k_i = 2 \times (i+1) + 1$. When $i < n-1$, we use the channel direction weights extracted by k_i to guide the adjacent scale kernel k_{i-1}. The S'_{n-1} with the largest receptive field, which reflects the overall characteristics of images, is directly connected to the next stage. For each S'_i, the weights of channels can be computed as

$$\mathcal{W}_{S'_i} = \sigma\left(\text{Conv1D}\left(g(S'_i)\right)\right) \tag{8}$$

where

$$g(S'_i) = \frac{1}{H \cdot W}\sum_{h=1}^{H}\sum_{w=1}^{W} S'_i(h, w). \tag{9}$$

Here, σ is a sigmoid function, Conv1-D indicates 1-D convolution with kernel size m, $g(S'_i)$ is channelwise global average pooling, and $\mathcal{W}_{S'_i} \in \mathbb{R}^{1\times 1\times C^*}$ is the attention weight. When the input has higher (lower) channel dimensionality, there are longer (shorter) range interactions between channels [64]. Therefore, the kernel size m should be varied according to the channel dimension to achieve proper cross-channel interaction. Wang et al. [65] proposed the proportional nonlinear mapping relationship between the kernel scale and the channel dimension. Because the channel dimension is the power of 2, a fundamental relationship between C^* and m can be expressed as $C^* = 2^{(m\times a - b)}$. We set parameters a and b to 1 and 2, respectively. Correspondingly, m is calculated as

$$m = |\ (\log_2(C^*) + 1)/2\ |_{\text{odd}} \tag{10}$$

where $|x|_{\text{odd}}$ indicates the nearest odd number of x.

Afterward, we obtain the channel-wise weight from the preprocessed multiscale feature map and conduct information interaction between the attention weight vectors of different scales. Mathematically, the output of cross-channel attention can be represented as

$$Z'_i = \begin{cases} S'_i, & i = n-1 \\ \mathcal{W}_{S'_i} \circ S'_{i+1}, & i = 0, 1, 2, \ldots, n-2. \end{cases} \tag{11}$$

Here, $\circ$ is the element-wise product, $Z'_i \in \mathbb{R}^{H\times W\times C^*}$ refers to feature maps with channel attention weights guided by different scale receptive fields. Next, we extract the entire multiscale features in a cascade mode to realize the interaction of information and fuse the cross-attention weighted feature maps without destroying the original channel attention vectors. We adopt the more efficient concatenation operator instead of the summation operator because it strengthens the complete feature representation and maintains the information of feature maps. After going through the entire multiscale channel attention module, the final output $Z_{\text{out}} \in \mathbb{R}^{H\times W\times C}$ can be expressed as

$$Z_{\text{out}} = \text{Concat}([Z'_0, Z'_1, Z'_2, \ldots, Z'_{n-1}]). \tag{12}$$

As illustrated by the above analysis, the proposed MSFF module can effectively extract multiscale spatial information at a finer-grained level. At the same time, it can realize richer multiscale feature representations and focus on enhancing cross-dimensional channel attention. As a result, the MSFF module is flexibly deployed in the proposed backbone and develops a long-range channel dependency.

E. SA Attention and Scale Convolution

As mentioned above, each stage of DCNNet contains a classification branch, which mainly consists of a SA attention and a scale convolution block. As presented in Fig. 4, SA attention uses a lightweight bottom-up and top-down feedforward structure to effectively enhance the different representations of the target position under multiple receptive fields. It first expands the receptive field by downsampling the convolutional layers. Then, a bilinear upsampling convolutional layer is attached after batch normalization and ReLU activation function, and a sigmoid activation is appended to obtain an attention map between 0 and 1. Finally, the upsampled attention map and the original feature map are subjected to a dot product operation. Using various receptive field information to guide the transformation process in the original feature space, SA attention

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

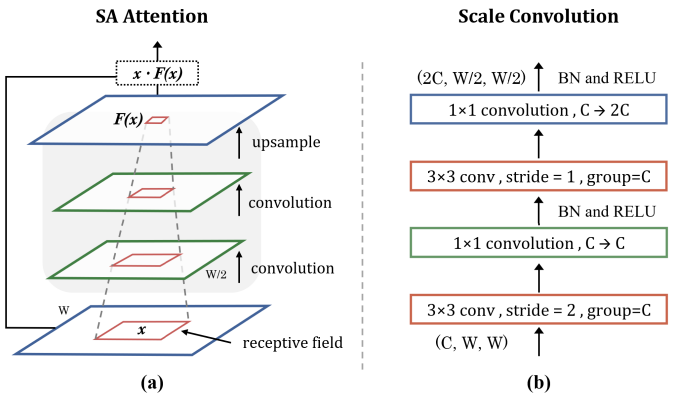

Fig. 4. (a) Details of the SA attention. (b) Scale convolution modules of *Stage_1* and *Stage_2*.

further improves the accuracy of each classification branch. In addition, the scale convolution consists of a basic bottleneck block to align the feature dimensions of each branch with the deepest one. The scale convolution of the deepest branch does not require feature alignment and thus consists of an average pooling layer.

IV. Experiments and Analysis

A. Dataset Description

We conduct experiments on three publicly available RS scene classification datasets to verify the effectiveness of our framework. Three scene datasets include the North Western Polytechnical University (NWPU)-RESISC45 Dataset [66], Aerial Image Dataset (AID) [67], and the UC Merced (UCM) dataset [68]. They are all commonly used benchmarks, which can represent large, medium, and small data volumes respectively. At the same time, to verify the robustness of our proposed method, we also performed the fine-grained classification as an ablation experiment on the FAIR1M dataset. The descriptions and settings of these four datasets in experiments are as follows.

1) NWPU-RESISC45: NWPU is a large-scale RS scene classification benchmark dataset obtained from Google Earth. The dataset consists of 31 500 satellite images divided into 45 categories. Each category includes 700 RGB spatial images of size 256 × 256 with spatial resolutions ranging from 30 to 0.2 m. Consistent with the original article [66], we only use 10% and 20% of the samples to train the generated network.

2) AID: The AID dataset is jointly proposed by Wuhan University and Huazhong University of Science and Technology in 2017. It contains 10 000 images distributed in 30 categories. Each category consists of 220 to 400 images. The size of each image is 600 × 600 pixels in RGB space, with spatial resolutions ranging from 8 to 0.5 m. The imbalanced distribution of classes, minor interclass variance, and higher intraclass variance make the classification of AID datasets more challenging. According to the document of AID [67], we adjust the ratio of the training set to 20% and 50%.

3) UCM: The UC Merced Land-Use Dataset is one of the most classic RS scene classification datasets. It is released in 2010 and includes 2100 images at 1-foot pixel resolution from the United States Geological Survey (USGS) National Map. The dataset is divided into 21 scene categories, and each category contains 100 images. The size of each image is 256 × 256 pixels. Furthermore, we use 50% and 80% of the images as the training set and the remaining 50% and 20% as the test set in our experiments.

4) FAIR1M: The FAIR1M dataset is proposed in 2021 and contains RS images with a resolution of 0.3 m to 0.8 m distributed in multiple countries and regions. It annotates five categories and 37 sub-categories according to oriented bounding boxes. We select three categories of "Airplane," "Ship," and "Vehicle" in the FAIR1M dataset, and a total of 27 subcategories are selected to conduct fine-grained classification experiments. These categories are more difficult to classify because of high inter-class similarity and within-class variation. We expand each category to 3000 images for a total of 81 000 images, and use 60% and 80% of the images as the training set and the remaining 40% and 20% as the test set in our experiments.

As a single-source dataset with a small amount of data, UCM is relatively simple to classify. In contrast, the AID and NWPU datasets are multisource RS datasets that are difficult to classify, and the images in the datasets come from different sensors with the complex spatial distribution and spectral effects. In the experiments, we randomly crop the images of the NWPU and UCM datasets to 224 × 224 pixels, and crop the AID images with the original size of 600 × 600 to 448 × 448 pixels. To further improve the generalization ability of the model, we perform image augmentation through random vertical and horizontal flipping, image rotation, image padding, and illumination adjustment. The same dataset setup is guaranteed in the experiments.

B. Implementation Details

Table I shows the corresponding module structure when images are output from different *Stages*. When the network is fed an original image with a scale of $H \times W \times 3$, it first goes through a 7 × 7 convolutional layer with stride 4 for downsampling and a 3 × 3 max-pooling layer with stride 2, where the quantity H, W represent the height and width, respectively. Moreover, we simply adjust the extended dimension to further reduce the passed parameters between different partition stages. The size of feature maps passed from *Stage_1* to *Stage_2* is reduced to two-thirds of original images, and the size of feature maps passed from *Stage_2* to *Stage_3* is reduced to one-third of the original images.

For the base parameter settings, we choose momentum stochastic gradient descent with cosine scheduling. The momentum is set to 0.9 during gradient vector optimization so that the gradient can be smoothed. The weight decay is set to 5×10^{-4} to prevent over-fitting. Meanwhile, the initial learning rate is set to 0.1, which is divided by a factor of 10 at the one-third and two-thirds of the total epochs, respectively. To reduce experimental errors, we perform each experiment five times and report the results as the mean and standard deviation of the five experiments. All experiments are implemented on an RTX-3090 GPU with PyTorch 1.8.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILED ARCHITECTURE AND PARAMETERS OF DCNNET

(256,256,3)	**Exit from *Stage_1* (Terminal)**			**Exit from *Stage_2* (Cloud-initial)**			**Exit from *Stage_3* (Cloud-final)**		
Output Size	Layer	Block setting	Params	Layer	Block setting	Params	Layer	Block setting	Params
(32,32,64)	7×7, 64, stride = 4 & 3×3, max-pool, stride = 2								
(32,32,128)	**Block-1**	$\begin{pmatrix}1\times1,\ 64\\ \text{MSFF},\ 64\\ 1\times1,128\end{pmatrix}\times3$	0.123 M 1.171 %	Block-1	$\begin{pmatrix}1\times1,\ 64\\ \text{MSFF},\ 64\\ 1\times1,128\end{pmatrix}\times3$	0.087 M 0.828 %	Block-1	$\begin{pmatrix}1\times1,\ 64\\ \text{MSFF},\ 64\\ 1\times1,128\end{pmatrix}\times3$	0.087 M 0.828 %
(16,16,256)	SC-T1	$\begin{pmatrix}3^2\text{ sep},128\\ 1\times1,\ 256\\ \text{groups} = 128\end{pmatrix}\times1$	0.052 M 0.500 %	**Block-2**	$\begin{pmatrix}1\times1,128\\ \text{MSFF},128\\ 1\times1,256\end{pmatrix}\times3$	0.417 M 3.989 %	Block-2	$\begin{pmatrix}1\times1,128\\ \text{MSFF},128\\ 1\times1,256\end{pmatrix}\times3$	0.280 M 2.676 %
(8,8,512)	SC-T2	$\begin{pmatrix}3^2\text{ sep},256\\ 1\times1,\ 512\\ \text{groups} = 256\end{pmatrix}\times1$	0.203 M 1.941 %	SC-E1	$\begin{pmatrix}3^2\text{ sep},256\\ 1\times1,\ 512\\ \text{groups} = 256\end{pmatrix}\times1$	0.203 M 1.941 %	Block-3a	$\begin{pmatrix}1\times1,256\\ \text{MSFF},256\\ 1\times1,512\end{pmatrix}\times9$	2.822 M 27.012 %
(4,4,1024)	SC-T3	$\begin{pmatrix}3^2\text{ sep},512\\ 1\times1,1024\\ \text{groups} = 512\end{pmatrix}\times1$	0.799 M 7.655 %	SC-E2	$\begin{pmatrix}3^2\text{ sep},512\\ 1\times1,1024\\ \text{groups} = 512\end{pmatrix}\times1$	0.799 M 7.655 %	**Block-3b**	$\begin{pmatrix}1\times1,512\\ \text{MSFF},512\\ 1\times1,1024\end{pmatrix}\times3$	4.211 M 40.303 %
——	3 × 3, average pool, fc								
Computational complexity	0.11 G FLOPs , 20.37%			0.15 G FLOPs , 27.78%			0.28 G FLOPs , 51.85%		

[1] Block represents the feature extractor of each stage, and the feature extractor of *Stage_3* consists of Block-3a and Block-3b.

[2] The highlighted feature extraction modules (Block-1, Block-2 and Block-3b) contain the SA attention.

[3] SC represents the scale convolution module. The scale convolution module of *Stage_3* is only composed of a 4×4 average pooling layer, so it is omitted in this table.

C. Evaluation Metrics

In the experiments, we evaluate the performance of DCNNet from three aspects: accuracy index, parameter quantities and time cost, and class confusion matrix.

1) Accuracy Index: Considering the specificity of the distributed model and the operability of actual deployment, we use different accuracy measures for each exit point of the model. Accuracy metrics are introduced as follows.

1) *Terminal Accuracy* is the accuracy when all samples are withdrawn from *Stage_1*.
2) *Cloud-initial Accuracy* is the accuracy when all samples are withdrawn from *Stage_2*.
3) *Cloud-final Accuracy* is the accuracy when all samples are withdrawn from *Stage_3*.
4) *Overall Accuracy* refers to the average accuracy of samples output in the three branch classifiers after introducing the progressive inference mechanism. The percentage of samples classified at each exit point is determined by the entropy threshold $\mathcal{E}^v$.
5) *Ensemble Accuracy* combines predictions from the classification branches of different hierarchies, and it can be seen as the highest accuracy that the overall model can achieve. We weight the probability vectors of each branch classifier, and the label corresponding to the largest value in the output vector is the fusion decision.

The calculation process of the ensemble accuracy can be expressed as follows. During inference, we compute the information entropy $M^v = -\hat{p}^v \log(\hat{p}^v)$ of the probability vector $\hat{p}^v$. M^v is used to measure the uncertainty of the sample's prediction on the vth branch. Therefore, we increase the weight α^v to reduce the classification contribution of branches with low confidence, which is calculated as

$$\alpha^v = \frac{\exp(-M^v)}{\sum_{v=1}^{\mathcal{V}} \exp(-M^v)}. \tag{13}$$

By multiplying the obtained weight vector α_v and $\hat{p}^v$, the ensemble probability vector of the sample on all branches can be expressed as $\sum_{v=1}^{\mathcal{V}} \alpha^v \hat{p}^v$. Thus, the predicted label $\hat{y}$ is denoted as

$$\hat{y} = \underset{v=1,2,\ldots,\mathcal{V}}{\operatorname{argmax}} \left(\sum_{v=1}^{\mathcal{V}} \alpha^v \hat{p}^v\right). \tag{14}$$

Consequently, the ensemble accuracy calculates the quotient of correct samples and all samples according to the label.

2) Parameter Quantities and Time Cost: We evaluate the efficiency of the overall model by the amount of parameters and time cost. At the same time, we also apply the number of floating point operations (FLOPs) to evaluate the computational complexity of the model, which is often used as an indirect measure of neural network speed. In addition, the experiments evaluate the time cost of DCNNet by recording the model inference time of single-input and multi-inputs, respectively.

3) Confusion Matrix: The confusion matrix is one of the most basic and intuitive methods to measure the accuracy of classification models. In the confusion matrix, each element is generated by counting the test images according to the actual class and the predicted class. Rows represent actual categories and columns represent predicted categories.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH BENCHMARK METHODS FOR RS SCENE CLASSIFICATION

Methods \ OA(%)	NWPU		AID		UCM		Index	
	10%	20%	20%	50%	50%	80%	Params(M)	FLOPs(G)
AlexNet [27]	81.77±0.48	85.13±0.33	84.98±0.59	86.88±0.56	88.14±0.75	89.26±0.57	60.10	0.77
VGG16 [28]	87.93±0.39	90.14±0.31	87.01±0.43	89.86±0.62	90.28±0.55	91.13±0.42	138.37	15.53
GoogLeNet [29]	82.54±0.55	86.43±0.27	83.91±0.35	86.35±0.25	90.36±0.36	90.97±0.43	13.81	1.40
Inception-V3 [69]	90.66±0.32	91.61±0.51	90.41±0.48	91.57±0.33	93.45±0.33	95.33±0.42	27.16	2.85
Xception [45]	89.96±0.36	91.30±0.49	90.60±0.48	91.93±0.22	93.96±0.27	95.40±0.25	22.86	8.42
ResNet-50 [30]	91.09±0.33	92.57±0.50	91.95±0.41	93.23±0.39	93.61±0.31	96.42±0.21	25.56	4.12
SE-ResNet-50 [34]	93.77±0.26	94.38±0.29	93.05±0.21	94.90±0.38	94.80±0.36	97.97±0.45	28.09	3.90
SE-ResNext-50 [70]	93.37±0.41	94.57±0.20	93.25±0.39	95.81±0.28	**95.89±0.28**	98.15±0.35	27.56	4.28
DenseNet169 [71]	90.63±0.43	91.85±0.35	91.88±0.32	93.05±0.20	91.31±0.39	94.58±0.46	14.15	3.42
SqueezeNet1.0 [46]	84.73±0.42	86.14±0.45	86.82±0.46	87.23±0,30	88.12±0.41	91.93±0.36	1.25	0.83
MobileNetV2 [43]	87.02±0.38	89.13±0.26	88.23±0.24	90.73±0.28	91.11±0.32	93.32±0.18	3.50	0.33
ShuffleNet-1.5x [47]	85.15±0.26	86.53±0.23	86.15±0.30	88.89±0.24	88.77±0.41	91.28±0.30	2.92	0.31
EfficientNet-B0 [72]	88.94±0.33	90.03±0.21	90.19±0.29	92.41±0.37	91.46±0.31	94.25±0.40	5.32	0.39
Terminal *(ours)*	90.03±0.28	91.97±0.17	91.58±0.26	93.62±0.31	92.39±0.26	95.20±0.34	1.17	0.11
Cloud-initial *(ours)*	92.61±0.24	93.11±0.19	92.91±0.23	94.68±0.12	93.12±0.44	97.36±0.38	1.51	0.15
Cloud-final *(ours)*	**94.19±0.26**	**95.21±0.22**	**93.64±0.25**	**96.23±0.19**	95.21±0.31	**98.87±0.26**	7.40	0.28

D. Scene Classification Experiments

In Table II, we compare the experimental results of the proposed method with some classical classification methods on three benchmark datasets (UCM, AID, and NWPU). The evaluation indicators include two aspects: model complexity indicators and accuracy indicators. The complexity indicators refer to the number of parameters and FLOPs. And the three accuracy indicators of DCNNet in the table represent terminal accuracy, cloud-initial accuracy, and cloud-final accuracy, respectively. The highest precision value in each column of the table is highlighted in bold. Among the listed lightweight methods, EfficientNet-B0 [72] has the best results. The parameter amount of our terminal model is only about one-fifth of that of EfficientNet-B0, and the FLOPs is only about 28.2%. Still, it achieves an accuracy improvement of about 1% on all three datasets, and the accuracy of cloud-initial model is improved by 2% to 3%. In addition, SE-ResNext-50 [46], [70] has the best performance among the compared methods. Compared with SE-ResNext-50, our cloud-final model achieves a similar level of accuracy while reducing the number of parameters by about 73%.

In addition to comparing some advanced lightweight model methods (SqueezeNet [46], MobileNet [43], ShuffleNet [47], and EfficientNet [72]), we also compared several prevalent attention-based methods, which includes CNN attention methods such as BiMobileNet [49], APDC-Net [39], LCNN-BFF [48], DFAGCN [73], and strategies that combine attention with vision transformer (ViT) [74] such as ViT+PA, ViT+SE, and SCViT [75]. The comparison with these state-of-the-art methods as shown in Table III and the best results are highlighted in bold. For the citation results, we adopted the same data-processing methods and division ratios as the comparison methods. The metric DCNNet-EA represents the ensemble accuracy of our proposed method.

1) Results on NWPU: Compared with the BiMobileNet, our method improves the accuracy by 2.52% and 1.72% at training set ratios of 10% and 20%, respectively. SCViT proposes a spatial channel feature-preserving method suitable for RS scene classification, obtaining the best results among ViT-based methods. In contrast, the DCNNet under joint training improves the accuracy by 1.86% and 1.14%. We demonstrate that the proposed model has significant gain over accuracy.

2) Results on AID: When the ratios of the training set are 20% and 50%, the ensemble accuracies of the DCNNet model are 94.94% and 97.34%. Compared with the BiMobileNet, the proposed method achieves similar accuracy at lower training ratios. DCNNet has slightly higher accuracy and minor variance, the practical effect is stable, and the accuracy is improved by 0.47% at higher training ratios. Compared with the ViT series methods, the accuracy of the proposed method is slightly higher than the original ViT model at a 20% training rate, and the accuracy of DCNNet is 0.36% higher than the highest accuracy SCViT at a 50% training rate. It is worth noting that compared with SCViT-B, our model parameters are only about one-third, and the FLOPs are about 16%.

3) Results on UCM: We set the ratio of training data to 50% and 80%, and the proposed model achieves 98.76% and 99.35% accuracy, respectively. Since the UCM dataset has simple data categories and low intraclass variances, the latest methods have achieved promising experimental results.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON WITH STATE-OF-THE-ART METHODS

Model	Year	NWPU(%)		AID(%)		UCM(%)	
		tr=10%	tr=20%	tr=20%	tr=50%	tr=50%	tr=80%
BiMobileNet† [49]	2020	92.06±0.14	94.08±0.11	94.83±0.24	96.87±0.23	98.45±0.27	99.03±0.28
APDC-Net† [39]	2020	85.94±0.22	87.84±0.26	88.56±0.29	92.15±0.29	95.01±0.43	97.05±0.43
LCNN-BFF† [48]	2020	86.53±0.15	91.73±0.17	91.66±0.48	94.62±0.16	94.64±0.21	99.29±0.24
DFAGCN† [73]	2021	—	89.29±0.28	—	94.88±0.22	—	98.48±0.42
ViT [74]	2021	91.38±0.12	93.87±0.19	94.85±0.25	96.32±0.31	98.35±0.28	98.87±0.26
ViT+PA† [75]	2021	92.65±0.20	94.24±0.16	95.31±0.11	96.72±0.16	98.58±0.31	99.14±0.27
ViT+SE† [75]	2021	92.39±0.21	94.60±0.17	95.43±0.22	95.61±0.57	98.55±0.20	99.10±0.20
SCViT† [75]	2022	92.72±0.04	94.66±0.10	**95.56±0.17**	96.98±0.16	**98.90±0.19**	**99.57±0.31**
DCNNet-EA($ours$)	2022	**94.58±0.18**	**95.80±0.12**	94.94±0.16	**97.34±0.18**	98.76±0.27	99.35±0.32

†: We refer the number reported in [75].

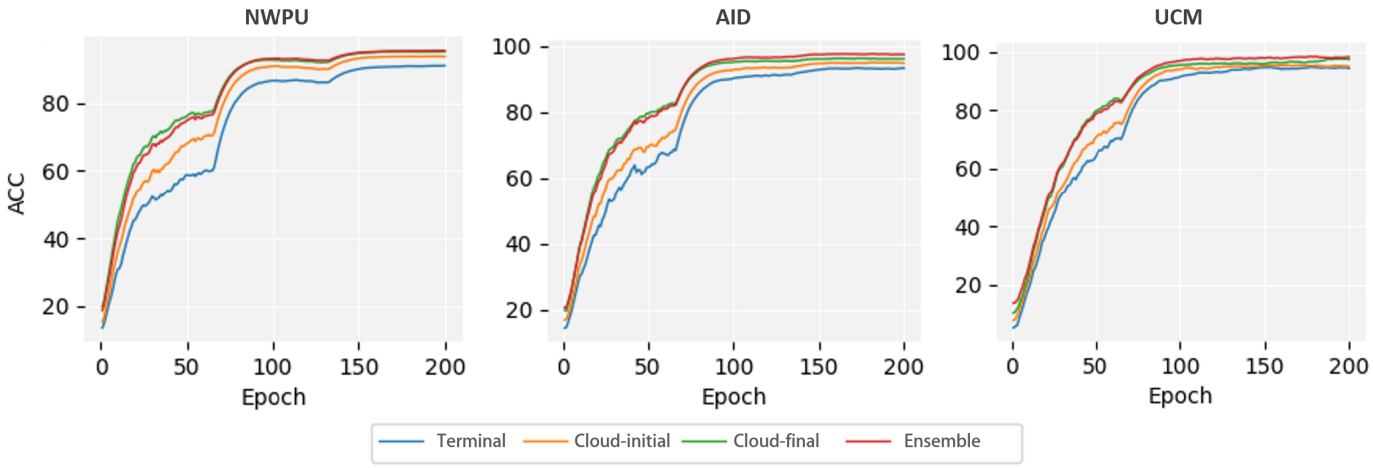

Fig. 5. Training curves on three datasets. The proportions of the training set are 20% NWPU, 50% AID, and 80% UCM. The learning rate changes at one-third and two-thirds epochs.

However, the accuracy of the DCNNet model is still higher than most methods, slightly less than the highest accuracy of the SCViT model.

Fig. 5 displays the curve of the classification accuracy over the test set for the DCNNet model during training. The inflection points of each curve reflect the change in the learning rate. In the early stage of training, the shallower branch has not yet learned the features of the deeper network, and the accuracy of the exit branches of terminal and cloud-initial has a particular gap compared with the cloud-final. The accuracy curve of cloud tends to converge in the later stages of training. In the process of back-propagating the gradient vector, the deeper network can guide the shallower network through self-distillation, thereby further improving the accuracy of the terminal classifiers.

Fig. 6 visualizes the classification accuracy-parameter plots for each model. Overall, compared with classical lightweight models and the latest advanced methods, DCNNet achieves similar classification accuracy as the best methods with fewer parameters and FLOPs. Meanwhile, the improvement on the NWPU dataset is more significant than that on the AID and UCM datasets. This indicates that DCNNet can obtain higher confidence performance on datasets with more categories and higher complexity. This is because the distillation between the three branches and the proposed feature extraction backbone and attention modules play essential roles in this process. We will demonstrate the impact of these strategies in the ablation studies in the next section.

E. Ablation Studies

We conduct a series of ablation experiments to analyze the contribution of each component in the proposed method. First, we set corresponding entropy thresholds for different classifier exit points, and compared the overall classification accuracy and speedup under different thresholds. Based on this, we demonstrate the trade-off between efficiency and accuracy provided by the progressive inference mechanism.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

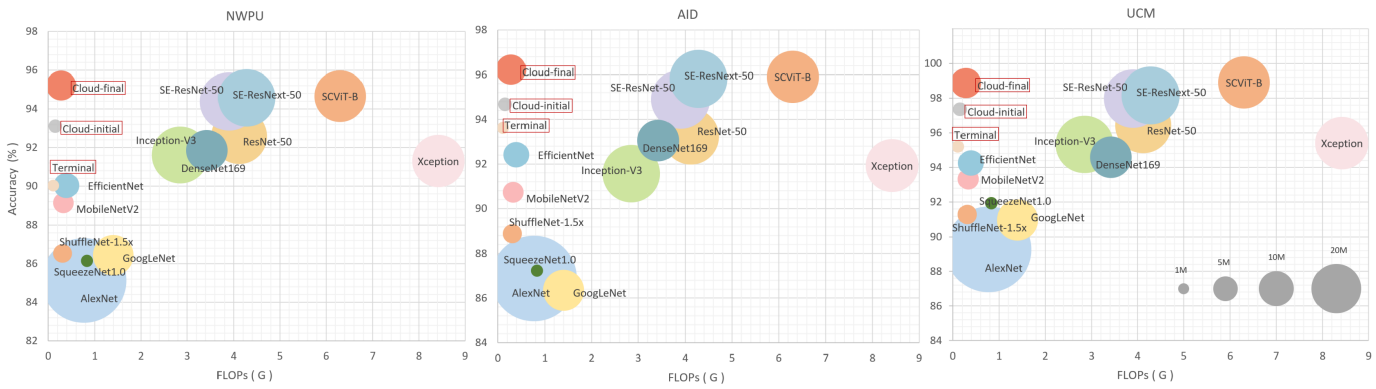

Fig. 6. Comparison diagrams of FLOPs-Accuracy with classical methods. The sizes of the circles represent the model parameters.

TABLE IV
ABLATION STUDIES OF PROGRESSIVE INFERENCE. SPEEDUP METRICS ARE MEASURED BY FLOPS

Thresholds	NWPU			AID			UCM		
	Exit Ratio(%)	OA	Speedup	Exit Ratio(%)	OA	Speedup	Exit Ratio(%)	OA	Speedup
[0.00, 1.00]	All-Stage1	91.97%	4.91×	All-Stage1	93.62%	4.91×	All-Stage1	95.20%	4.91×
[1.00, 0.00]	All-Stage2	93.11%	3.59×	All-Stage2	94.68%	3.59×	All-Stage2	97.36%	3.59×
[1.00, 1.00]	All-Stage3	95.21%	1.00×	All-Stage3	96.23%	1.00×	All-Stage3	98.87%	1.00×
[0.88, 0.90]	(91.1 5.10 3.80)	92.13%	4.21×	(94.6 4.10 1.30)	93.79%	4.60×	(96.2 2.70 1.10)	95.83%	4.66×
[0.90, 0.92]	(81.6 13.5 4.90)	92.54%	3.96×	(91.5 4.80 3.70)	94.16%	4.22×	(94.1 3.51 2.39)	96.24%	4.45×
[0.92, 0.93]	(58.6 33.4 8.00)	93.15%	3.42×	(79.9 16.1 4.20)	94.62%	4.01×	(93.2 4.10 2.70)	96.26%	4.38×
[0.94, 0.95]	(34.4 43.7 21.9)	94.40%	2.43×	(44.1 48.3 7.60)	94.91%	3.33×	(74.2 18.5 7.29)	96.68%	3.63×
[0.95, 0.96]	(18.8 31.9 49.3)	95.01%	1.61×	(25.6 41.9 32.5)	95.45%	2.03×	(47.3 45.6 7.10)	97.33%	3.40×
[0.96, 0.97]	(6.50 14.0 79.5)	95.15%	1.18×	(47.3 45.6 7.10)	95.98%	1.52×	(25.4 52.1 22.5)	97.72%	2.37×

Second, we verify the effect of the proposed distributed self-distillation on the accuracy of the three-stage classifiers, especially the terminal classifiers. Thirdly, we evaluate the performance of the MSFF from the aspects of model accuracy, FLOPs, and parameters amount. Next, we compared the accuracy improvement after configuring SA attention on each classification branch. Finally, we present the classification accuracy of the proposed method on 27 fine-grained categories in the FAIR1M dataset and demonstrate the robustness of the DCNNet model. For a fair comparison, all the experiments are conducted under the same settings and training strategies.

1) Effect of Progressive Inference: Considering model acceleration and avoiding redundant operations in the inference process, we introduce the progressive inference mechanism and design a normalized confidence entropy threshold to achieve the early classification of terminals. We select nine sets of thresholds for the first and second exit points. When the entropy score calculated from the confidence of the current sample reaches or exceeds the given threshold, the sample is allowed to complete the inference. Finally, the deepest classifier outputs the remaining samples that do not meet the exit conditions of the previous two branches. Table IV shows the overall accuracy, the proportion of data exited by the three classification branches, and the speedup of DCNNet.

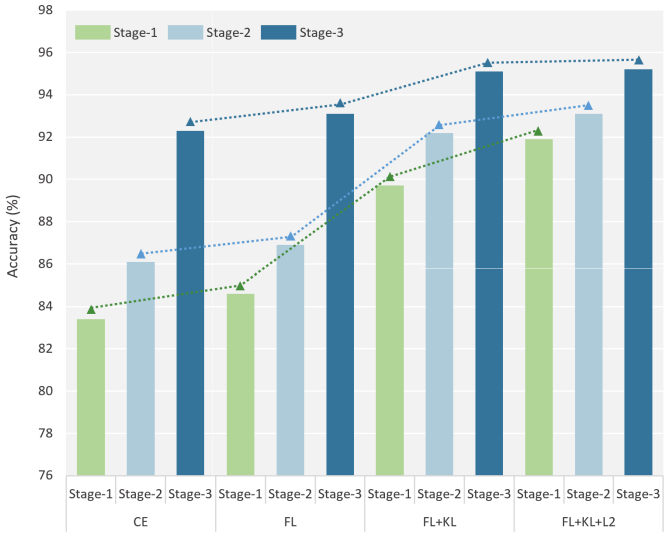

Fig. 7. Effect of distributed self-distillation on the classification accuracy of three-stage classifiers.

Among them, the exit ratio indicator represents the proportion of test images output from three branches, respectively. The speedup index is based on the network without the progressive inference mechanism. In the RS scene classification task, DCNNet can achieve a speedup of 4.91 times. The sacrifice

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

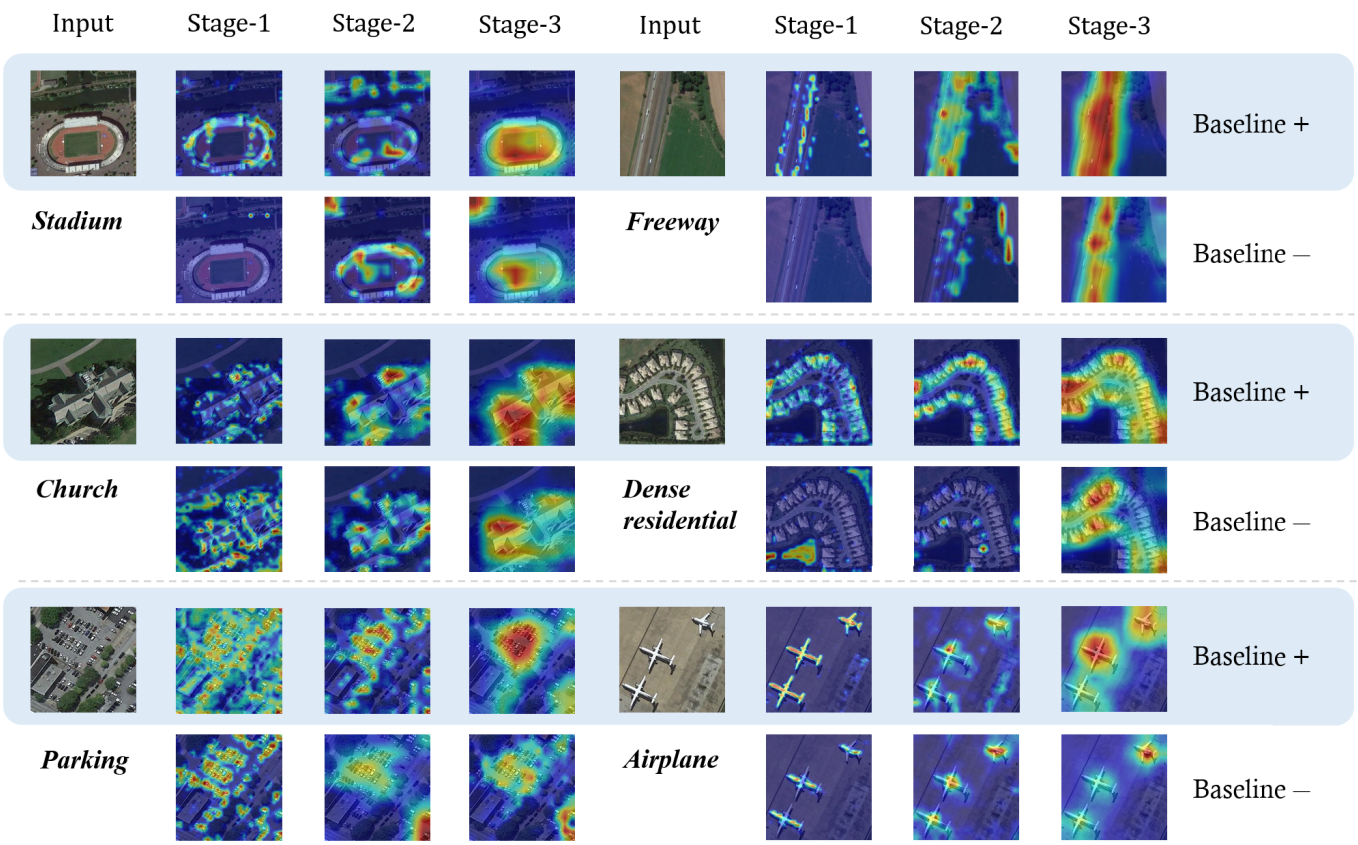

Fig. 8. Comparison of visualized heatmaps. We randomly select six images from the NWPU dataset as representatives and show the focus of Baseline+ and Baseline- after "Stage-1," "Stage-2," and "Stage-3."

of the average accuracy to obtain the highest speedup is about 3.17% on the three datasets. The experimental results prove that the progressive inference mechanism can substantially make a valuable trade-off strategy between accuracy and efficiency.

2) Effect of Distributed Self-Distillation: We transfer the knowledge representation ability of the cloud (teacher) classifiers to the terminal (student) classifier by introducing self-distillation. Unlike traditional knowledge distillation frameworks, which often require extensive experiments to find the best teacher model architecture, our method does not need to introduce additional complex network structures. The distributed self-distillation not only accelerates the parameter convergence of DCNNet during training, but also significantly improves the classification accuracy of terminal networks.

The proposed distributed self-distillation consists of three parts. The first is the distillation between feature maps of the same dimension. This is followed by KL divergence distillation between the softmax layers at each stage. The last part is the multiclass FL between labels and each classifier. Fig. 7 compares the accuracy metrics of the three-stage classifiers under different loss functions on the NWPU dataset. Replacing the cross-entropy (CE) loss with a multiclass FL improves the classification accuracy of the three classifiers by an average of 0.93% without the distillation strategy. At this time, the cloud-final classifier (Stage-3) can achieve an accuracy of 93.12%. However, the accuracy of the terminal (Stage-1) and cloud-initial (Stage-2) classifiers are only 84.68% and 86.91%. After introducing the KL divergence loss and the L2 loss, the performance of the terminal network has been greatly improved. The accuracy of the terminal classifiers is increased by 5.3% and 2.2%, respectively. The accuracy of the cloud-initial classifier is increased by 5.1% and 0.9%. Although the gain is small, the accuracy of the cloud-final classifier is also improved by 2.1%. In joint distillation, smaller models tend to yield more gains. This feature is more conducive to the distributed deployment of terminal networks in the inference process.

3) Effect of Multiscale Feature Fusion: The MSFF module is mainly composed of two parts: the PSC module and cross-channel attention. The PSC module is used to provide multiscale feature representation. Cross-channel attention strengthens the assignment of the most informative feature expressions from the channel direction while suppressing less useful feature expressions, allowing the model to attend to important regions within a context adaptively. To reduce the overall network complexity and establish long-range channel dependencies, we utilize the MSFF module for replacing the 3×3 convolution of the classical bottleneck layer, and the formed block serves as the main unit of the backbone. The network incorporating the MSFF module and the one using only 3×3 convolutions are denoted as Baseline^{+} and Baseline^{-}, respectively.

Ablation reports of the MSFF module are shown in Table V. We compare the ensemble accuracy of the three classifiers on the NWPU, AID, and UCM datasets. From the experimental results, we can see that using the MSFF module instead of the ordinary 3×3 convolution has achieved an

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ABLATION STUDIES OF MSFF MODULE

	NWPU-EA	AID-EA	UCM-EA	Params	FLOPs
Baseline$^-$	92.31%	94.56%	96.87%	15.71M	0.90G
Baseline$^+$	95.80%	97.34%	99.35%	9.85M	0.54G
Comparison	3.49% ↑	2.78% ↑	2.48% ↑	5.86M ↓	0.36G ↓

TABLE VI
ABLATION STUDIES OF SA ATTENTION

	Terminal	Cloud-initial	Cloud-final	DCNNet-EA
SA$^-$	90.15%	91.48%	94.32%	94.54%
SA$^+$	91.97%	93.11%	95.21%	95.80%
Comparison	1.82% ↑	1.63% ↑	0.89% ↑	1.26% ↑

TABLE VII
COMPARISON WITH BENCHMARK METHODS ON THE FAIR1M DATASET

Model	Accuracy(%)	
	tr=60%	tr=80%
AlexNet [27]	83.42±0.51	86.12±0.40
VGG16 [28]	86.31±0.32	88.54±0.28
GoogLeNet [29]	84.50±0.45	86.27±0.30
ResNet-50 [30]	88.02±0.25	90.95±0.12
ResNext-50 [70]	90.90±0.48	92.77±0.36
MobileNetV2 [43]	86.62±0.49	88.36±0.42
ShuffleNet-1.5x [47]	84.13±0.71	87.92±0.52
Terminal $(ours)$	89.90±0.30	90.17±0.36
Cloud-initial $(ours)$	90.66±0.35	91.80±0.32
Cloud-final $(ours)$	93.58±0.24	94.82±0.23
DCNNet-EA $(ours)$	**93.77±0.12**	**95.20±0.26**

accuracy improvement of 3.49% on the NWPU dataset. The classification accuracy on the AID and UCM datasets is also improved by 2.78% and 2.48%, respectively. At the same time, the MSFF module makes reasonable use of grouped convolutions and depthwise separable convolutions to avoid redundant computations, which reduces the overall network parameters by 5.86M and FLOPs by 0.36G.

Fig. 8 shows the class activation maps of Baseline$^+$ and Baseline$^-$ in Stage-1, Stage-2, and Stage-3, which intuitively illustrates the effect of the MSFF module. We randomly selected six categories of images in the NWPU dataset. Their respective class activation maps use different color depths to reflect the learning focus of the network at each stage. It can be seen that after introducing the MSFF module, the classification effect of the network at each stage is significantly better than the original bottleneck structure. In early Stage-1 and Stage-2, the proposed network can provide a more confident judgment on the input category. In the last stage, the results of Baseline$^+$ can more comprehensively cover the critical part

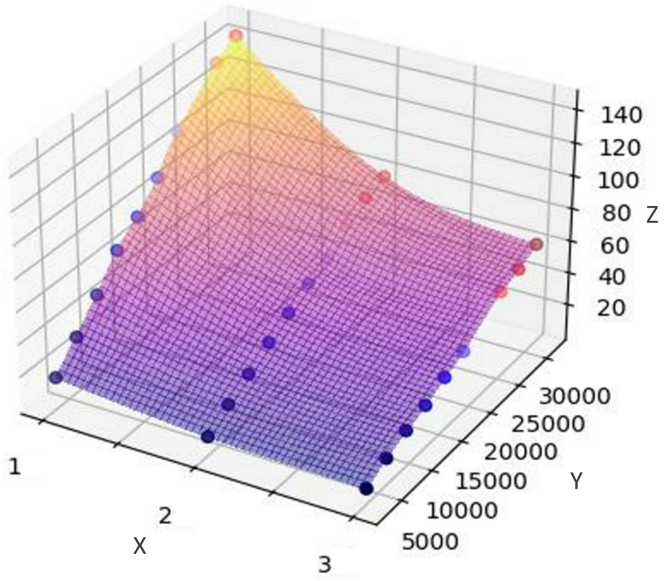

Fig. 9. Inference timeliness verification of multiple inputs under different data scales. The x-axis represents the number of terminals, the y-axis represents the number of samples to be classified, and the z-axis represents the inference time (s).

of the RS scene while reducing excess interference in the complex background.

4) Effect of SA Attention: We design the SA attention on each branch to further improve the classification accuracy of DCNNet. Ablation studies of the SA attention are shown in Table VI. The network with and without the SA attention are denoted as SA$^+$ and SA$^-$, respectively. We compared the terminal, cloud-initial, cloud-final, and ensemble accuracy on the 20% training ratio NWPU dataset. The experimental results show that the corresponding accuracy is improved by 1.82%, 1.63%, 0.89%, and 1.26% after introducing SA attention.

5) Fine-Grained Classification Experiment: To further verify the robustness of the proposed model, we conducted classification experiments on 27 fine-grained classes that are more challenging for classification in the FAIR1M dataset. The experimental results are shown in Table VII. When the proportion of the training set is 60% and 80%, the terminal accuracy of DCNNet is 3.28% and 2.25% higher than MobileNetV2, and the cloud-final accuracy is 2.68% and 2.05% higher than ResNext-50. Meanwhile, the ensemble accuracy reaches 93.77% and 95.20%, which verifies the superiority of our method.

F. Visual Analysis

1) Multiple Input Experiment: The above experimental results demonstrate the excellent performance of DCNNet in terms of accuracy and efficiency. To meet the requirements of practical applications, we extend the number of terminals to test the inference efficiency of distributed network with multiple inputs. A certain proportion of images are randomly selected in the test set of the NWPU-RESISC45 dataset and averaged to each terminal. The shared cloud model uniformly collects samples that do not reach the output threshold and continue inference. Fig. 9 shows the inference time of the DCNNet model when the number of terminals

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

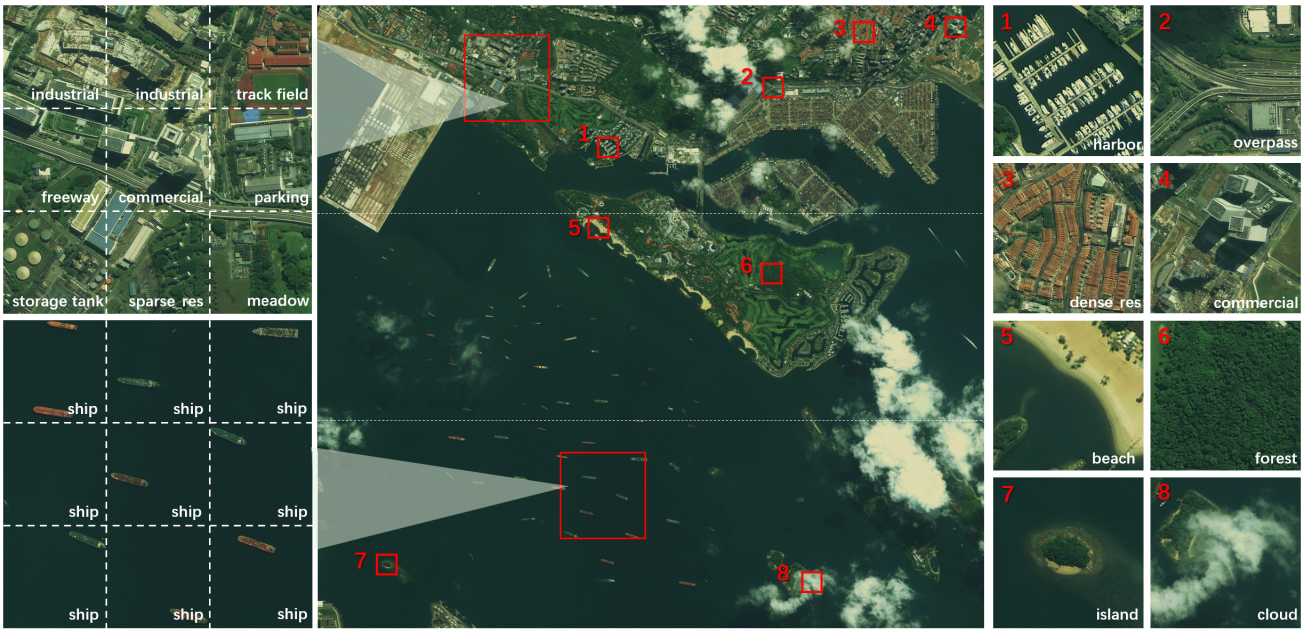

Fig. 10. Visualization of sliding-window classification results in the large-scale RS image using three terminals. The picture is divided into three horizontal parts, which are processed by three terminals. Classification results for several slices are presented on both sides. The right side presents the 256 × 256 sliding windows, and the left side shows the area of nine slices.

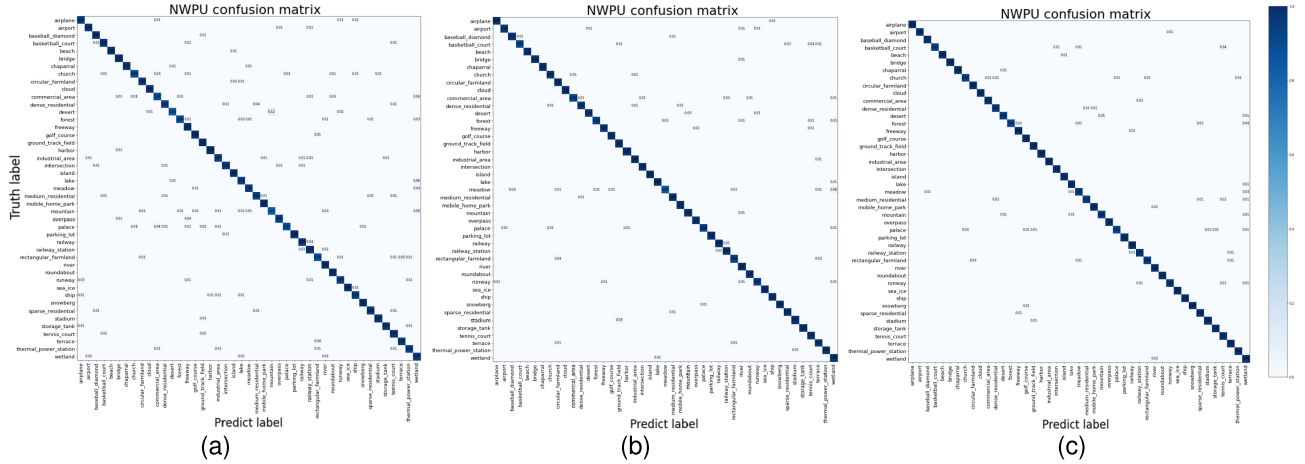

Fig. 11. Confusion matrices on the NWPU dataset. (a) Terminal. (b) Cloud-initial. (c) Cloud-final.

varies from 1 to 3, and the number of input images varies from 3150 to 31500. From Fig. 9, it can be seen that compared with a single input, multiple inputs can significantly improve the efficiency of the model. Meanwhile, we use three terminals to classify each 256 × 256 sliding window on a large RS image with a resolution of 10858 × 10147. The visualization results of some areas are presented in Fig. 10. In the experimental process, compared with a single terminal, the processing time of three terminals achieves a 2.32× speedup ratio.

2) Confusion Matrix: To comprehensively evaluate the performance of the proposed DCNNet, we use confusion matrices to show the intersection of predicted and truth labels. Figs. 11–13 show the confusion matrices generated by the three-stage DCNNet classifiers on the NWPU-RESISC45, AID, and UCM datasets, respectively. Here, we do not set the thresholds for the progressive inference mechanism, which means that the images detected by each classifier are consistent in the same dataset. From the results of confusion matrices, we can make the following analyses.

Firstly, from the diagonal elements, the proposed method performs well in most classes. Some classes can achieve 100% accuracy, and the rest also achieve reliable classification performance. At the same time, with the increase of network depth, the classification accuracy is gradually improved, and the number of misclassified samples is gradually reduced. For example, the category of "industrial-area" in the NWPU-RESISC45 dataset obtained 93%, 99%, and 100% classification accuracy in three stages, respectively. The "commercial-area" has classification accuracy of 88%, 92%, and 97%. The accuracy of "parking" in the AID dataset is 95%, 98%, and 100%. And the accuracy of "intersection" in the UCM dataset is 98%, 100%, and 100%

Secondly, for very few categories, an interesting phenomenon is that the classification accuracy of cloud-initial is slightly higher than the cloud-final. For example, we observed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

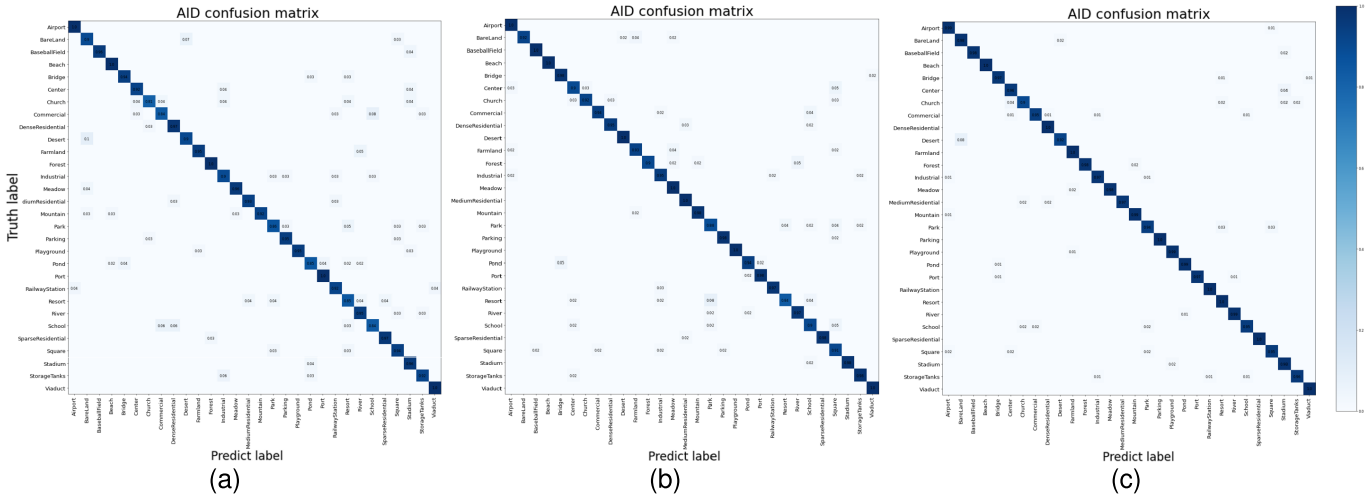

Fig. 12. Confusion matrices on the AID dataset. (a) Terminal. (b) Cloud-initial. (c) Cloud-final.

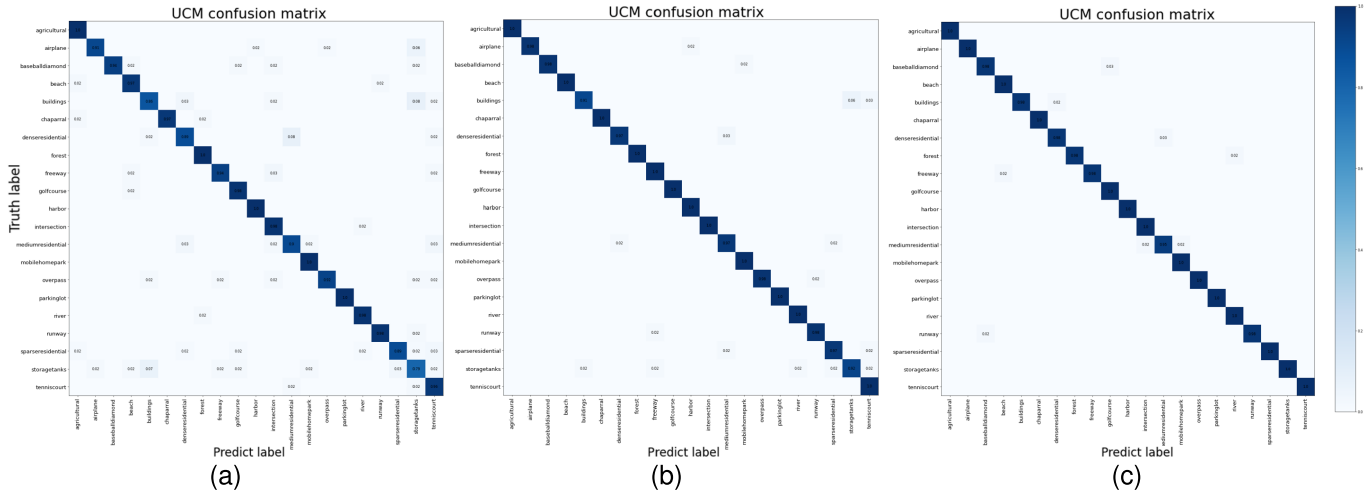

Fig. 13. Confusion matrices on the UCM dataset. (a) Terminal. (b) Cloud-initial. (c) Cloud-final.

that the classification accuracy of "highways" in the UCM dataset was 94%, 100%, and 98%, respectively. This result suggests that for a small number of easy-to-classify categories, their representations in early layers are sufficient for correct classification. On the contrary, the final classifier representation may be distracted by overly complex or irrelevant features that do not generalize well and lead to "overthinking." Nevertheless, when the progressive inference mechanism is introduced, the image that reaches the threshold can be output without further reasoning. The progressive inference mechanism can circumvent the "overthinking" problem to a certain extent, so there can be a relatively stable accuracy improvement in the process of terminal-cloud inference.

V. Conclusion

In this article, we propose a distributed neural network framework DCNNet for multiplatform RS application requirements. DCNNet achieves a better trade-off between model efficiency and accuracy than existing cloud-only and terminal-only methods. Comparisons with existing state-of-the-art methods on four benchmark RS datasets, NWPU, AID, UCM, and FAIR1M, demonstrate the superiority of our method. In future work, the distributed framework can be extended in various applications, such as RS object detection and semantic segmentation. However, the proposed method still has shortcomings under changing bandwidth and unstable connections, and we plan to further optimize the process of feature transmission according to the communication environment. Additionally, we will investigate the performance of the distributed framework when inputting multiple types of RS data.

References

[1] Y. Gu, T. Liu, X. Jia, and J. A. Benediktsson, "Nonlinear multiple kernel learning with multiple-structure-element extended morphological profiles for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3235–3247, Jun. 2016.

[2] G. Cheng, X. Xie, J. Han, L. Guo, and G.-S. Xia, "Remote sensing image scene classification meets deep learning: Challenges, methods, benchmarks, and opportunities," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, no. 99, pp. 3735–3756, Jun. 2020.

[3] Z. Huang, W. Li, X.-G. Xia, X. Wu, Z. Cai, and R. Tao, "A novel nonlocal-aware pyramid and multiscale multitask refinement detector for object detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–20, 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

[4] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Apr. 2019.

[5] Z. Lv, W. Shi, X. Zhang, and J. A. Benediktsson, "Landslide inventory mapping from bitemporal high-resolution remote sensing images using change detection and multiscale segmentation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 5, pp. 1520–1532, Mar. 2018.

[6] D. Hong, J. Hu, J. Yao, J. Chanussot, and X. X. Zhu, "Multimodal remote sensing benchmark datasets for land cover classification with a shared and specific feature learning model," *ISPRS J. Photogramm. Remote Sens.*, vol. 178, pp. 68–80, Aug. 2021.

[7] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.

[8] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, 2020, pp. 1–15.

[9] Z. Lu, S. Rallapalli, K. Chan, S. Pu, and T. L. Porta, "Augur: Modeling the resource requirements of ConvNets on mobile devices," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 352–365, Feb. 2021.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[11] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," 2020, *arXiv:2002.06440*.

[12] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 4387–4398.

[13] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 328–339.

[14] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 560–569.

[15] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–11.

[16] G. Chen, X. Song, H. Zeng, and S. Jiang, "Scene recognition with prototype-agnostic scene layout," *IEEE Trans. Image Process.*, vol. 29, pp. 5877–5888, 2020.

[17] Y. Yang and S. Newsam, "Geographic image retrieval using local invariant features," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 2, pp. 818–832, Feb. 2013.

[18] V. Risojević and Z. Babić, "Fusion of global and local descriptors for remote sensing image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 4, pp. 836–840, Jul. 2013.

[19] G. Cheng, Z. Li, X. Yao, L. Guo, and Z. Wei, "Remote sensing image scene classification using bag of convolutional features," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 10, pp. 1735–1739, Oct. 2017.

[20] K. Nogueira, O. A. B. Penatti, and J. A. dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognit.*, vol. 61, pp. 539–556, Jan. 2017.

[21] E. Li, J. Xia, P. Du, C. Lin, and A. Samat, "Integrating multilayer features of convolutional neural networks for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5653–5665, Oct. 2017.

[22] W. Han, R. Feng, L. Wang, and Y. Cheng, "A semi-supervised generative framework with deep learning features for high-resolution remote sensing image scene classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 23–43, Nov. 2018.

[23] N. Yang, H. Tang, H. Sun, and X. Yang, "DropBand: A simple and effective method for promoting the scene classification accuracy of convolutional neural networks for VHR remote sensing imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 257–261, Feb. 2018.

[24] G. Cheng, C. Yang, X. Yao, L. Guo, and J. Han, "When deep learning meets metric learning: Remote sensing image scene classification via learning discriminative CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 5, pp. 2811–2821, May 2018.

[25] J. Wang, W. Liu, L. Ma, H. Chen, and L. Chen, "IORN: An effective remote sensing image scene classification framework," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 11, pp. 1695–1699, Nov. 2018.

[26] F. Hu, G.-S. Xia, J. Hu, and L. Zhang, "Transferring deep convolutional neural networks for the scene classification of high-resolution remote sensing imagery," *Remote Sens.*, vol. 7, no. 11, pp. 14680–14707, 2015.

[27] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 2, pp. 84–90, Jun. 2012.

[28] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[29] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 1–9.

[30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[31] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-capsnet," *Remote Sens.*, vol. 11, no. 5, p. 494, 2019.

[32] J. Chen et al., "Contextual information-preserved architecture learning for remote-sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2021.

[33] J. Li, M. Weinmann, X. Sun, W. Diao, Y. Feng, and K. Fu, "Random topology and random multiscale mapping: An automated design of multiscale and lightweight neural network for remote-sensing image recognition," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2021.

[34] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[35] F. Wang et al., "Residual attention network for image classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 3156–3164.

[36] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1155–1167, Sep. 2019.

[37] B. Fang, Y. Li, H. Zhang, and J. C.-W. Chan, "Hyperspectral images classification based on dense convolutional networks with spectral-wise attention mechanism," *Remote Sens.*, vol. 11, no. 2, p. 159, 2019.

[38] R. Zhu, L. Yan, N. Mo, and Y. Liu, "Attention-based deep feature fusion for the scene classification of high-resolution remote sensing images," *Remote Sens.*, vol. 11, no. 17, p. 1996, Jan. 2019.

[39] Q. Bi, K. Qin, H. Zhang, J. Xie, and K. Xu, "APDC-Net: Attention pooling-based convolutional network for aerial scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 9, pp. 1603–1607, Sep. 2019.

[40] Y. Zhang, W. Li, M. Zhang, Y. Qu, R. Tao, and H. Qi, "Topological structure and semantic information transfer network for cross-scene hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Sep. 16, 2021, doi: 10.1109/TNNLS.2021.3109872.

[41] M. Zhang, W. Li, Y. Zhang, R. Tao, and Q. Du, "Hyperspectral and LiDAR data classification based on structural optimization transmission," *IEEE Trans. Cybern.*, early access, May 13, 2022, doi: 10.1109/TCYB.2022.3169773.

[42] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[43] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.

[44] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1580–1589.

[45] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1251–1258.

[46] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," 2016, *arXiv:1602.07360*.

[47] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 6848–6856.

[48] C. Shi, T. Wang, and L. Wang, "Branch feature fusion convolution network for remote sensing scene classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 5194–5210, 2020.

[49] D. Yu, Q. Xu, H. Guo, C. Zhao, Y. Lin, and D. Li, "An efficient and lightweight convolutional neural network for remote sensing image scene classification," *Sensors*, vol. 20, no. 7, p. 1999, Apr. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.

[50] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[51] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," 2016, *arXiv:1612.03928*.

[52] Y. Hou, Z. Ma, C. Liu, and C. C. Loy, "Learning lightweight lane detection CNNs by self attention distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1013–1021.

[53] L. Zhang, J. Song, A. Gao, J. Chen, C. Bao, and K. Ma, "Be your own teacher: Improve the performance of convolutional neural networks via self distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 3713–3722.

[54] M. Ji, S. Shin, S. Hwang, G. Park, and I.-C. Moon, "Refine myself by teaching myself: Feature refinement via self-knowledge distillation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 10664–10673.

[55] A. Graves, "Adaptive computation time for recurrent neural networks," 2016, *arXiv:1603.08983*.

[56] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser, "Universal transformers," 2018, *arXiv:1807.03819*.

[57] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, Dec. 2016, pp. 2464–2469.

[58] W. Liu, P. Zhou, Z. Zhao, Z. Wang, H. Deng, and Q. Ju, "FastBERT: A self-distilling BERT with adaptive inference time," 2020, *arXiv:2004.02178*.

[59] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "DeeBERT: Dynamic early exiting for accelerating BERT inference," 2020, *arXiv:2004.12993*.

[60] R. Schwartz, G. Stanovsky, S. Swayamdipta, J. Dodge, and N. A. Smith, "The right tool for the job: Matching model and instance complexities," 2020, *arXiv:2004.07453*.

[61] W. Zhou, C. Xu, T. Ge, J. McAuley, K. Xu, and F. Wei, "BERT loses patience: Fast and robust inference with early exit," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 18330–18341.

[62] C. He, M. Annavaram, and S. Avestimehr, "FedNAS: Federated deep learning via neural architecture search," in *Proc. CVPR Workshop Neural Archit. Search Beyond Represent. Learn.*, 2020.

[63] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[64] T. Zhang, G.-J. Qi, B. Xiao, and J. Wang, "Interleaved group convolutions," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 4373–4382.

[65] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2020, pp. 11534–11542.

[66] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[67] G. S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Apr. 2017.

[68] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2010, pp. 270–279.

[69] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.

[70] S. Xie, R. Girshick, P. Dollar, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1492–1500.

[71] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4700–4708.

[72] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.

[73] K. Xu, H. Huang, P. Deng, and Y. Li, "Deep feature aggregation framework driven by graph convolutional network for scene classification in remote sensing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5751–5765, Oct. 2022.

[74] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[75] P. Lv, W. Wu, Y. Zhong, F. Du, and L. Zhang, "SCViT: A spatial-channel feature preserving vision transformer for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4409512.

Ting Zhang received the B.Sc. degree from Tianjin University, Tianjin, China, in 2020. She is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China, and the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing.

Her research interests include computer vision and pattern recognition, especially on remote sensing image interpretation.

Zhirui Wang (Member, IEEE) received the B.Sc. degree from the Harbin Institute of Technology, Harbin, China, in 2013, and the Ph.D. degree from Tsinghua University, Beijing, China, in 2018.

He is currently an Assistant Researcher with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing. His research interests focus on the intelligent interpretation of remote sensing images, such as terrain classification, target detection, and recognition.

Peirui Cheng received the B.S. degree from the School of Information Science and Technology, University of Science and Technology of China, Hefei, China, in 2013, the M.S. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2016, and the Ph.D. degree from the School of Computer Science and Technology, University of Chinese Academy of Sciences, in 2022.

He is currently a Post-Doctoral Researcher with the Aerospace Information Research Institute, Chinese Academy of Sciences. His research interests mainly include scene text detection and remote sensing.

Guangluan Xu received the B.Sc. degree from Beijing Information Science and Technology University, Beijing, China, in 2000, and the M.Sc. and Ph.D. degrees (5-year Ph.D. Program) from the Institute of Electronics, Chinese Academy of Sciences, Beijing, in 2005.

He is currently a Professor with the Aerospace Information Research Institute, Chinese Academy of Sciences. His research interests include computer vision and remote sensing image understanding.

Xian Sun (Senior Member, IEEE) received the B.Sc. degree from the Beijing University of Aeronautics and Astronautics, Beijing, China, in 2004, and the M.Sc. and Ph.D. degrees from the Institute of Electronics, Chinese Academy of Sciences (CAS), Beijing, in 2009.

He was a Visiting Scholar with Karlsruher Institutfür Technologie, Karlsruhe, Germany, in 2013. He is currently a Professor with the Aerospace Information Research Institute, CAS. His research interests include computer vision, geospatial data mining, and remote sensing image understanding.

Dr. Sun was a recipient of the Outstanding Science and Technology Achievement Prize of the CAS in 2016 and the First Prize for The State Scientific and Technological Progress of China in 2019. He also serves as an Associate Editor for IEEE ACCESS and a Guest Editor for the special issue of the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS) and other journals.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:04:42 UTC from IEEE Xplore. Restrictions apply.